

No SIEM, No Problem – Real-World IR with Just Native Tools.

From RDP Attacks to Rogue Cron jobs - 5 days in the IR Trenches.

Analyst : LAO-TSE

Tool : WEV / UBUNTU

Focus : Host-based detection and incident response using native Linux and Windows tools.

Scenarios Investigated:

Day 11: RDP Brute Force – Failed Login Detection via Event Logs and PowerShell Containment

Day 12: Suspicious Bash Script – Process Discovery and Manual Removal from /tmp

Day 13: Malicious Cron Job – Persistence Detection and Safe Crontab Cleanup

Day 14: Suspicious PowerShell – Script Block Logging and Host File Restoration

Day 15: Outbound Beaconing – Network Connection Tracing and IP Blocking with UFW

Day 11: Detecting RDP Brute-Force Attacks {Windows}

Objective:

Simulate and detect a brute-force attack on RDP using Hydra, then respond using native Windows tools.

Tasks Performed :

Enabled RDP on a Windows machine

Used Hydra from Kali to perform a brute-force attack

Monitored Windows Event Viewer for Event ID 4625

Identified:

Logon Type: 10 (RemoteInteractive)

Failure reason: Bad password

Source IP: UBUNTU

Disabled the targeted user account using PowerShell

Blocked the attacker's IP using:

```
New-NetFirewallRule -DisplayName "Block Attacker" -Direction Inbound  
-RemoteAddress <Kali_IP> -Action Block
```

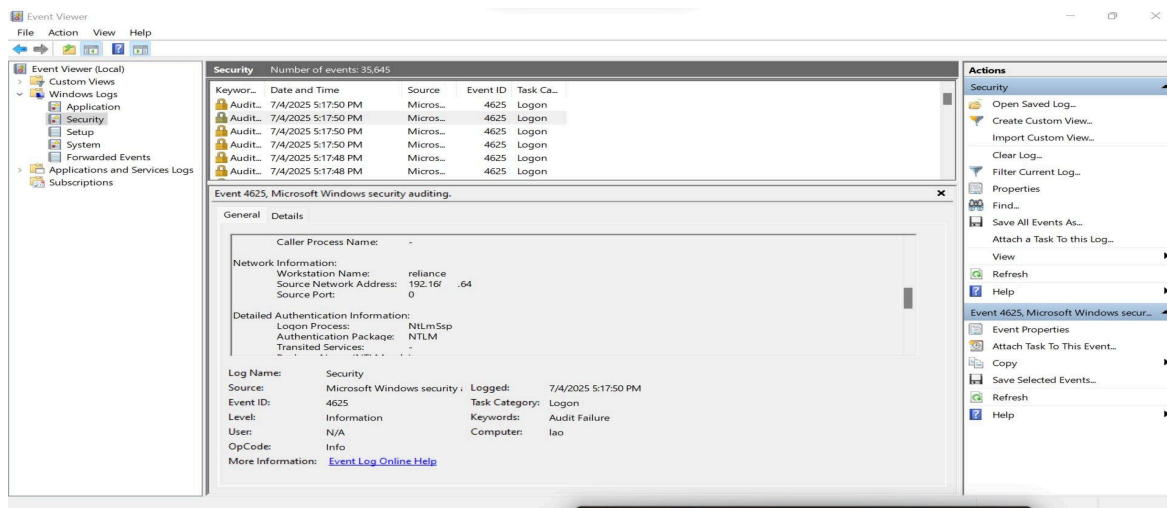
Exported logs for documentation

```
268 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Hydra is already the newest version (9.5-build3).
The following packages were automatically installed and are no longer required:
  libblvm1764 python3-netifaces
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 268 not upgraded.
root@reliance:/home/reliance# hydra --help
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra: invalid option -- :
root@reliance:/home/reliance# hydra -l lao -P password.txt rdp://[REDACTED]
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-07-04 17:16:31
[WARNING] rdp servers often don't like many connections, use -t 1 or -t 4 to reduce the number of parallel connections and -W 1 or -W 3 to wait between connection to allow the server to recover
[INFO] Reduced number of tasks to 4 (rdp does not like many parallel connections)
[WARNING] the rdp module is experimental. Please test, report - and if possible, fix.
[DATA] max 4 tasks per 1 server, 8 login tries (1:1/p:0), -2 tries per task
[DATA] attacking rdp://[REDACTED]
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-07-04 17:16:33
root@reliance:/home/reliance# hydra -l lao -P password.txt rdp://[REDACTED]
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-07-04 17:17:48
[WARNING] rdp servers often don't like many connections, use -t 1 or -t 4 to reduce the number of parallel connections and -W 1 or -W 3 to wait between connection to allow the server to recover
[INFO] Reduced number of tasks to 4 (rdp does not like many parallel connections)
[WARNING] the rdp module is experimental. Please test, report - and if possible, fix.
[DATA] max 4 tasks per 1 server, 8 login tries (1:1/p:0), -2 tries per task
[DATA] attacking rdp://[REDACTED]
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-07-04 17:17:50
root@reliance:/home/reliance#
```



Key Takeaways:

Event Viewer provides enough context to detect brute-force activity.

PowerShell can handle account lockdown and firewall rules.

Even without SIEM, attackers can be detected through pattern recognition in native logs.

Day 12: Suspicious Bash Script in /tmp (Linux)

Objective:

Detect and respond to a suspicious script mimicking malware in the /tmp directory on a Linux system.

Tasks Performed:

Planted fakebackup.sh in /tmp and ran it in the background

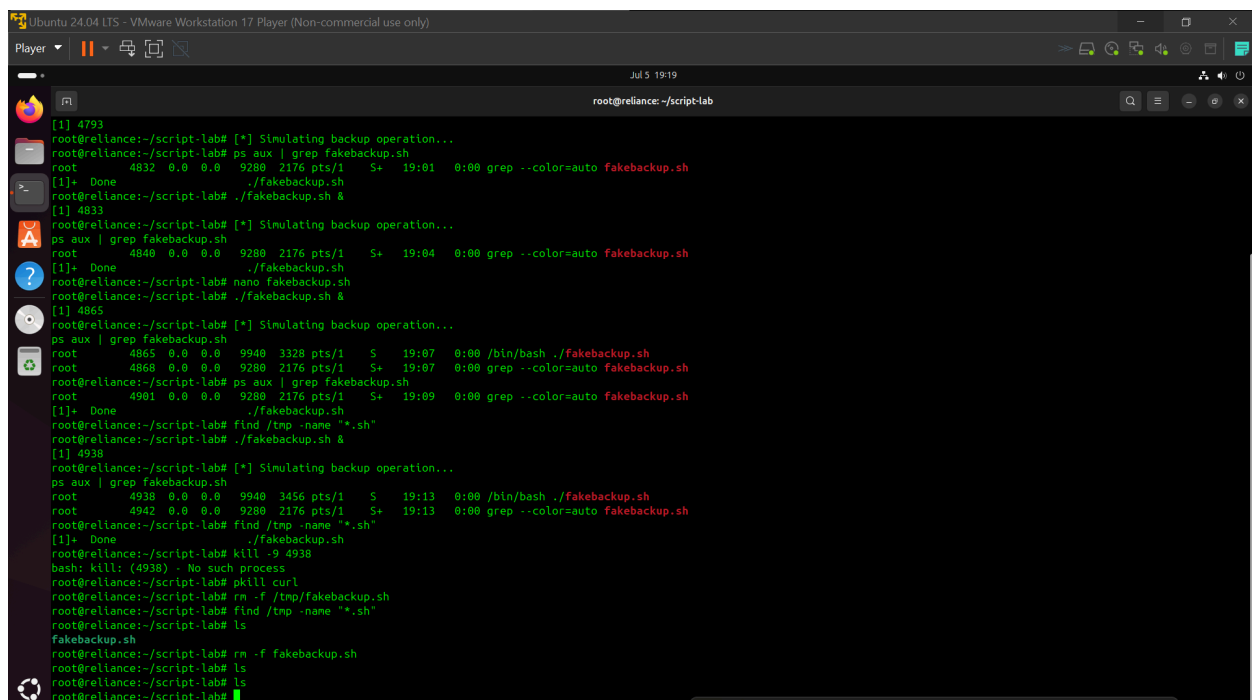
Found the process with `ps aux | grep fakebackup.sh`

Located the file with `find /tmp -name "*.sh"`

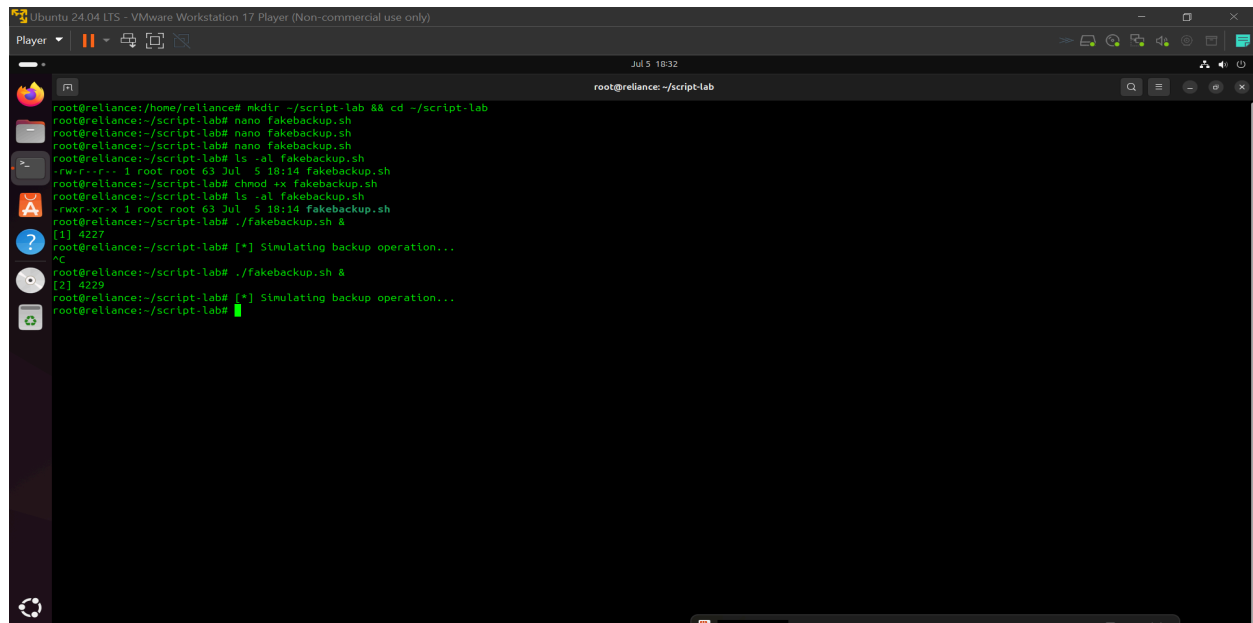
Terminated the process with `pkill curl`

Deleted the script

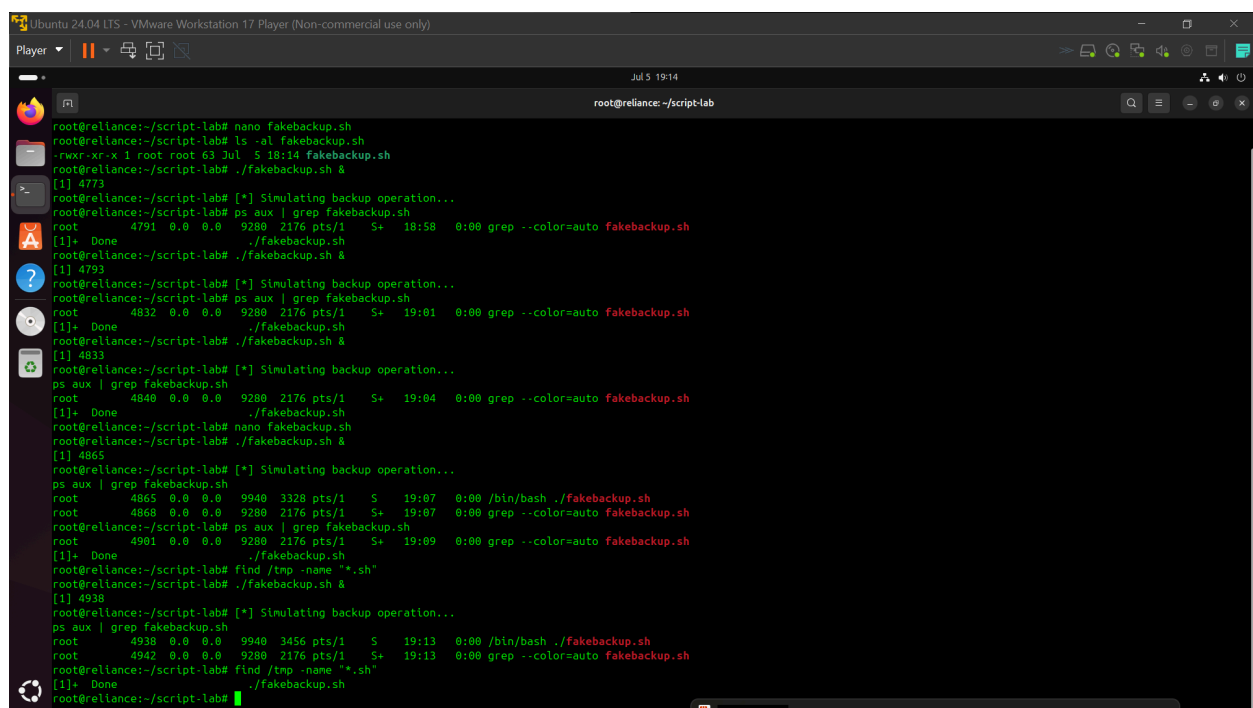
Checked `crontab -e` for persistence attempts



```
Ubuntu 24.04 LTS - VMware Workstation 17 Player (Non-commercial use only)
Player
Jul 5 19:19
root@reliance: ~/script-lab
[1] 4793
root@reliance:~/script-lab# [*] Simulating backup operation...
root@reliance:~/script-lab# ps aux | grep fakebackup.sh
root      4832  0.0  0.0   9280  2176 pts/1    S+   19:01   0:00 grep --color=auto fakebackup.sh
[1]+  Done                  ./fakebackup.sh
root@reliance:~/script-lab# ./fakebackup.sh &
[1] 4833
root@reliance:~/script-lab# [*] Simulating backup operation...
ps aux | grep fakebackup.sh
root      4840  0.0  0.0   9280  2176 pts/1    S+   19:04   0:00 grep --color=auto fakebackup.sh
[1]+  Done                  ./fakebackup.sh
root@reliance:~/script-lab# nano fakebackup.sh
root@reliance:~/script-lab# ./fakebackup.sh &
[1] 4865
root@reliance:~/script-lab# [*] Simulating backup operation...
ps aux | grep fakebackup.sh
root      4865  0.0  0.0   9940  3328 pts/1    S   19:07   0:00 /bin/bash ./fakebackup.sh
root      4868  0.0  0.0   9280  2176 pts/1    S+   19:07   0:00 grep --color=auto fakebackup.sh
root@reliance:~/script-lab# ps aux | grep fakebackup.sh
root      4901  0.0  0.0   9280  2176 pts/1    S+   19:09   0:00 grep --color=auto fakebackup.sh
[1]+  Done                  ./fakebackup.sh
root@reliance:~/script-lab# find /tmp -name "*.sh"
root@reliance:~/script-lab# ./fakebackup.sh &
[1] 4938
root@reliance:~/script-lab# [*] Simulating backup operation...
ps aux | grep fakebackup.sh
root      4938  0.0  0.0   9940  3456 pts/1    S   19:13   0:00 /bin/bash ./fakebackup.sh
root      4942  0.0  0.0   9280  2176 pts/1    S+   19:13   0:00 grep --color=auto fakebackup.sh
root@reliance:~/script-lab# find /tmp -name "*.sh"
root@reliance:~/script-lab# ./fakebackup.sh
[1]+  Done                  ./fakebackup.sh
root@reliance:~/script-lab# kill -9 4938
bash: kill: (4938) - No such process
root@reliance:~/script-lab# pkill curl
root@reliance:~/script-lab# rm -f /tmp/fakebackup.sh
root@reliance:~/script-lab# find /tmp -name "*.sh"
root@reliance:~/script-lab# ls
fakebackup.sh
root@reliance:~/script-lab# rm -f fakebackup.sh
root@reliance:~/script-lab# ls
root@reliance:~/script-lab# ls
root@reliance:~/script-lab#
```



```
root@reliance:/home/reliance# mkdir -p /script-lab && cd /script-lab
root@reliance:/script-lab# nano fakebackup.sh
root@reliance:/script-lab# nano fakebackup.sh
root@reliance:/script-lab# ls -al fakebackup.sh
-rw-r--r-- 1 root root 63 Jul  5 18:14 fakebackup.sh
root@reliance:/script-lab# chmod +x fakebackup.sh
root@reliance:/script-lab# ls -al fakebackup.sh
-rwxr-xr-x 1 root root 63 Jul  5 18:14 fakebackup.sh
root@reliance:/script-lab# ./fakebackup.sh &
[1] 4227
root@reliance:/script-lab# [*] Simulating backup operation...
^C
root@reliance:/script-lab# ./fakebackup.sh &
[2] 4229
root@reliance:/script-lab# [*] Simulating backup operation...
root@reliance:/script-lab#
```



```
root@reliance:/script-lab# nano fakebackup.sh
root@reliance:/script-lab# ls -al fakebackup.sh
-rwxr-xr-x 1 root root 63 Jul  5 18:14 fakebackup.sh
root@reliance:/script-lab# ./fakebackup.sh &
[1] 4773
root@reliance:/script-lab# [*] Simulating backup operation...
root@reliance:/script-lab# ps aux | grep fakebackup.sh
root      4791  0.0  0.0   9280  2176 pts/1    S+   18:58   0:00 grep --color=auto fakebackup.sh
[1]+  Done                  ./fakebackup.sh
root@reliance:/script-lab# ./fakebackup.sh &
[1] 4793
root@reliance:/script-lab# [*] Simulating backup operation...
root@reliance:/script-lab# ps aux | grep fakebackup.sh
root      4832  0.0  0.0   9280  2176 pts/1    S+   19:01   0:00 grep --color=auto fakebackup.sh
[1]+  Done                  ./fakebackup.sh
root@reliance:/script-lab# ./fakebackup.sh &
[1] 4833
root@reliance:/script-lab# [*] Simulating backup operation...
ps aux | grep fakebackup.sh
root      4840  0.0  0.0   9280  2176 pts/1    S+   19:04   0:00 grep --color=auto fakebackup.sh
[1]+  Done                  ./fakebackup.sh
root@reliance:/script-lab# nano fakebackup.sh
root@reliance:/script-lab# ./fakebackup.sh &
[1] 4865
root@reliance:/script-lab# [*] Simulating backup operation...
ps aux | grep fakebackup.sh
root      4865  0.0  0.0   9940  3328 pts/1    S   19:07   0:00 /bin/bash ./fakebackup.sh
root      4868  0.0  0.0   9280  2176 pts/1    S+   19:07   0:00 grep --color=auto fakebackup.sh
root@reliance:/script-lab# ps aux | grep fakebackup.sh
root      4901  0.0  0.0   9280  2176 pts/1    S+   19:09   0:00 grep --color=auto fakebackup.sh
[1]+  Done                  ./fakebackup.sh
root@reliance:/script-lab# find /tmp -name "*.sh"
root@reliance:/script-lab# ./fakebackup.sh &
[1] 4938
root@reliance:/script-lab# [*] Simulating backup operation...
ps aux | grep fakebackup.sh
root      4938  0.0  0.0   9940  3456 pts/1    S   19:13   0:00 /bin/bash ./fakebackup.sh
root      4942  0.0  0.0   9280  2176 pts/1    S+   19:13   0:00 grep --color=auto fakebackup.sh
root@reliance:/script-lab# find /tmp -name "*.sh"
[1]+  Done                  ./fakebackup.sh
root@reliance:/script-lab#
```

Key Takeaways:

/tmp is a common location for attacker scripts

Routine checks of /tmp and active processes can reveal hidden activity

It's essential to verify persistence mechanisms like cron jobs

Day 13: Malicious Cron Job (Linux)

Objective:

Simulate and investigate a malicious cron job designed to execute a script persistently on a Linux system.

Tasks Performed:

Created a script that logged to /tmp/.cron.log

Scheduled the script via root's crontab

Discovered the job using:

`crontab -l`

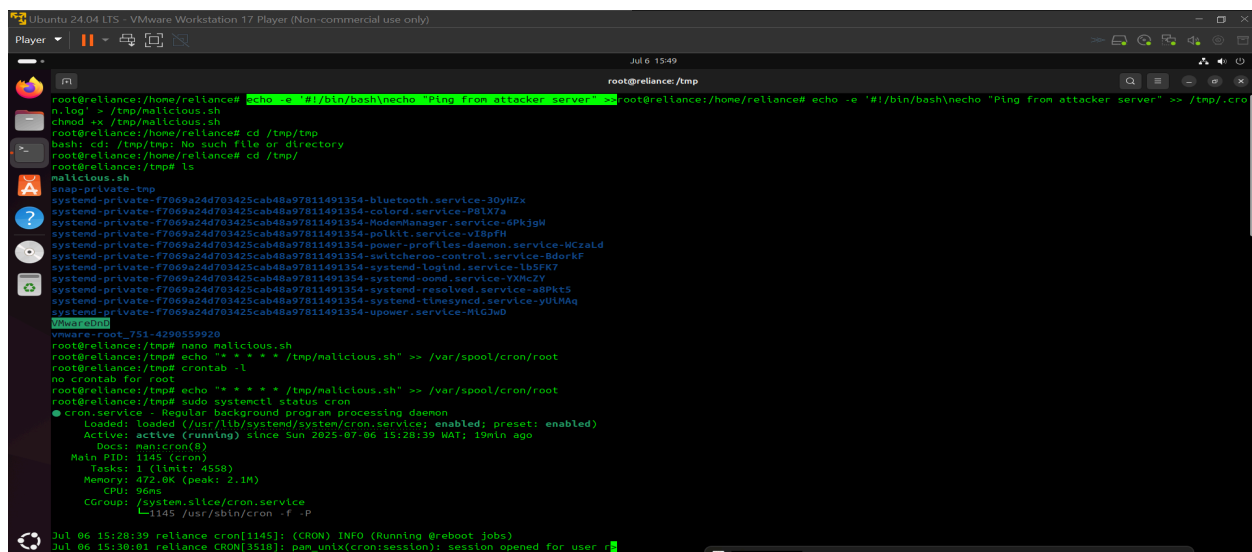
`grep -r "/tmp/" /etc/cron* /var/spool/cron`

Confirmed execution by reading the hidden log

Cleaned the crontab:

`crontab -l | grep -v "malicious.sh" | crontab -`

Deleted the malicious files and restarted the cron service



```
root@reliance:/home/reliance# echo -e '#!/bin/bash;necho "Ping from attacker server"' > /tmp/malicious.sh
root@reliance:/home/reliance# chmod +x /tmp/malicious.sh
root@reliance:/home/reliance# cd /tmp/
root@reliance:/tmp# ls
malicious.sh
root@reliance:/tmp# crontab -l
no crontab for root
root@reliance:/tmp# echo '* * * * * /tmp/malicious.sh' >> /var/spool/cron/root
root@reliance:/tmp# sudo systemctl status cron
● cron.service - Regular background program processing daemon
   Loaded: loaded (/usr/lib/systemd/system/cron.service; enabled; preset: enabled)
   Active: active (running) since Sun 2025-07-06 15:28:39 WAT; 19min ago
     Docs: man:cron(8)
    Main PID: 1145 (cron)
      Tasks: 1 (limit: 4558)
     Memory: 472.0K (peak: 2.1M)
        CPU: 96ms
      CGroup: /system.slice/cron.service
              └─1145 /usr/sbin/cron -f -p

Jul 06 15:28:39 reliance cron[1145]: (CRON) INFO (Running @reboot jobs)
Jul 06 15:30:01 reliance CRON[3510]: pam_unix(cron:session): session opened for user root
```

```
root@reliance:/tmp
Invalid mark letter (press RETURN)
root@reliance:/tmp# crontab -l
no crontab for root
root@reliance:/tmp# echo "* * * * * /tmp/malicious.sh" >> /var/spool/cron/root
root@reliance:/tmp# crontab -l
no crontab for root
root@reliance:/tmp# tail -f /tmp/root_cron_test.log
tail: cannot open '/tmp/root_cron_test.log' for reading: No such file or directory
root@reliance:/tmp# ls -l /tmp/malicious.sh
-rwxr-xr-x 1 root root 63 Jul 6 15:37 /tmp/malicious.sh
root@reliance:/tmp# echo -e '#!/bin/bash\n echo "Cron ran at $(date)" >> /tmp/root_cron_test.log' > /tmp/malicious.sh
chmod +x /tmp/malicious.sh
root@reliance:/tmp# crontab -e
no crontab for root - using an empty one
crontab: installing new crontab
root@reliance:/tmp# crontab -l
* * * * * /tmp/malicious.sh
root@reliance:/tmp# grep -r "/tmp/" /etc/cron* /var/spool/cron/crontabs
/var/spool/cron/crontabs/root: # (/tmp/crontab.SqWUCl/crontab installed on Sun Jul 6 15:59:30 2025)
/var/spool/cron/crontabs/root: * * * * * /tmp/malicious.sh
root@reliance:/tmp# cat /tmp/.cron.log
cat: /tmp/.cron.log: No such file or directory
root@reliance:/tmp# cd /tmp/
root@reliance:/tmp# ls
malicious.sh
root_cron_test.log
snap-private-tmp
systemd-private-f7069a24d703425cab48a97811491354-bluetooth.service-30yH2x
systemd-private-f7069a24d703425cab48a97811491354-colord.service-P8lX7a
systemd-private-f7069a24d703425cab48a97811491354-NodeManager.service-6PkjgM
systemd-private-f7069a24d703425cab48a97811491354-polkit.service-vI8pFH
systemd-private-f7069a24d703425cab48a97811491354-power-profiles-daemon.service-WCzald
systemd-private-f7069a24d703425cab48a97811491354-switcheroo-control.service-BdorkF
systemd-private-f7069a24d703425cab48a97811491354-systemd-logind.service-lb5FK7
systemd-private-f7069a24d703425cab48a97811491354-systemd-oond.service-YXMcZY
systemd-private-f7069a24d703425cab48a97811491354-systemd-resolved.service-a8Pkt5
systemd-private-f7069a24d703425cab48a97811491354-systemd-timesyncd.service-yUfMAq
systemd-private-f7069a24d703425cab48a97811491354-upower.service-MlGJwD

root@reliance:/tmp# cd /tmp/
root@reliance:/tmp# ls
malicious.sh
root_cron_test.log
snap-private-tmp
systemd-private-f7069a24d703425cab48a97811491354-bluetooth.service-30yH2x
systemd-private-f7069a24d703425cab48a97811491354-colord.service-P8lX7a
systemd-private-f7069a24d703425cab48a97811491354-NodeManager.service-6PkjgM
systemd-private-f7069a24d703425cab48a97811491354-polkit.service-vI8pFH
systemd-private-f7069a24d703425cab48a97811491354-power-profiles-daemon.service-WCzald
systemd-private-f7069a24d703425cab48a97811491354-switcheroo-control.service-BdorkF
systemd-private-f7069a24d703425cab48a97811491354-systemd-logind.service-lb5FK7
systemd-private-f7069a24d703425cab48a97811491354-systemd-oond.service-YXMcZY
systemd-private-f7069a24d703425cab48a97811491354-systemd-resolved.service-a8Pkt5
systemd-private-f7069a24d703425cab48a97811491354-systemd-timesyncd.service-yUfMAq
systemd-private-f7069a24d703425cab48a97811491354-upower.service-MlGJwD
VMware0n0
vmware-root_751-4290559920
root@reliance:/tmp# cat /tmp/malicious.sh
#!/bin/bash
echo "Cron ran at $(date)" >> /tmp/root_cron_test.log
root@reliance:/tmp# crontab -l | grep -v "malicious.sh" | crontab -
root@reliance:/tmp# rm -f /tmp/malicious.sh /tmp/.cron.log
root@reliance:/tmp# cd /tmp/
root@reliance:/tmp# ls
root_cron_test.log
snap-private-tmp
systemd-private-f7069a24d703425cab48a97811491354-bluetooth.service-30yH2x
systemd-private-f7069a24d703425cab48a97811491354-colord.service-P8lX7a
systemd-private-f7069a24d703425cab48a97811491354-NodeManager.service-6PkjgM
systemd-private-f7069a24d703425cab48a97811491354-polkit.service-vI8pFH
systemd-private-f7069a24d703425cab48a97811491354-power-profiles-daemon.service-WCzald
systemd-private-f7069a24d703425cab48a97811491354-switcheroo-control.service-BdorkF
systemd-private-f7069a24d703425cab48a97811491354-systemd-logind.service-lb5FK7
systemd-private-f7069a24d703425cab48a97811491354-systemd-oond.service-YXMcZY
systemd-private-f7069a24d703425cab48a97811491354-systemd-resolved.service-a8Pkt5
systemd-private-f7069a24d703425cab48a97811491354-systemd-timesyncd.service-yUfMAq
systemd-private-f7069a24d703425cab48a97811491354-upower.service-MlGJwD
VMware0n0
vmware-root_751-4290559920
root@reliance:/tmp#
```

Key Takeaways:

Cron jobs are easy to abuse for persistence

Searching cron directories and job listings is crucial during Linux IR

Cleanup must include both scheduled task and payload removal

Day 14: Suspicious PowerShell Activity (Windows)

Objective:

Simulate a suspicious PowerShell command and detect it using Windows logging tools.

Tasks Performed:

Enabled script block and module logging via Group Policy

Executed:

```
Start-Process "notepad.exe" -ArgumentList  
"C:\Windows\System32\drivers\etc\hosts"
```

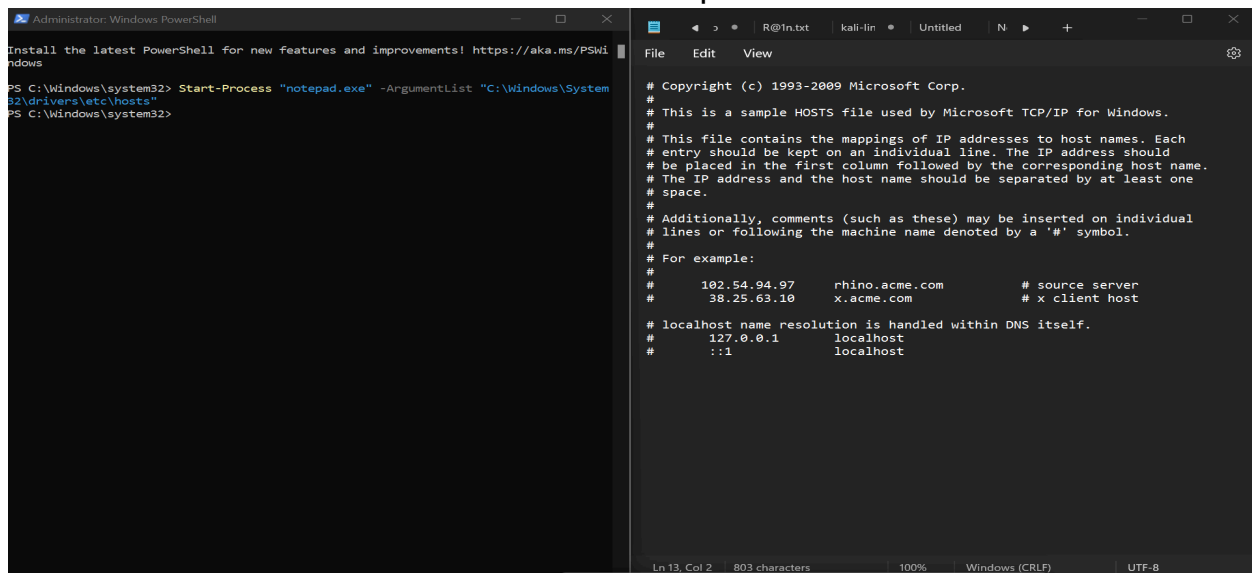
Analyzed Event ID 4103 in PowerShell logs

Blocked outbound connections:

```
New-NetFirewallRule -DisplayName "Block Network Access" -Direction  
Outbound -Action Block
```

Replaced the hosts file with a clean backup

Re-enabled the firewall rule after cleanup

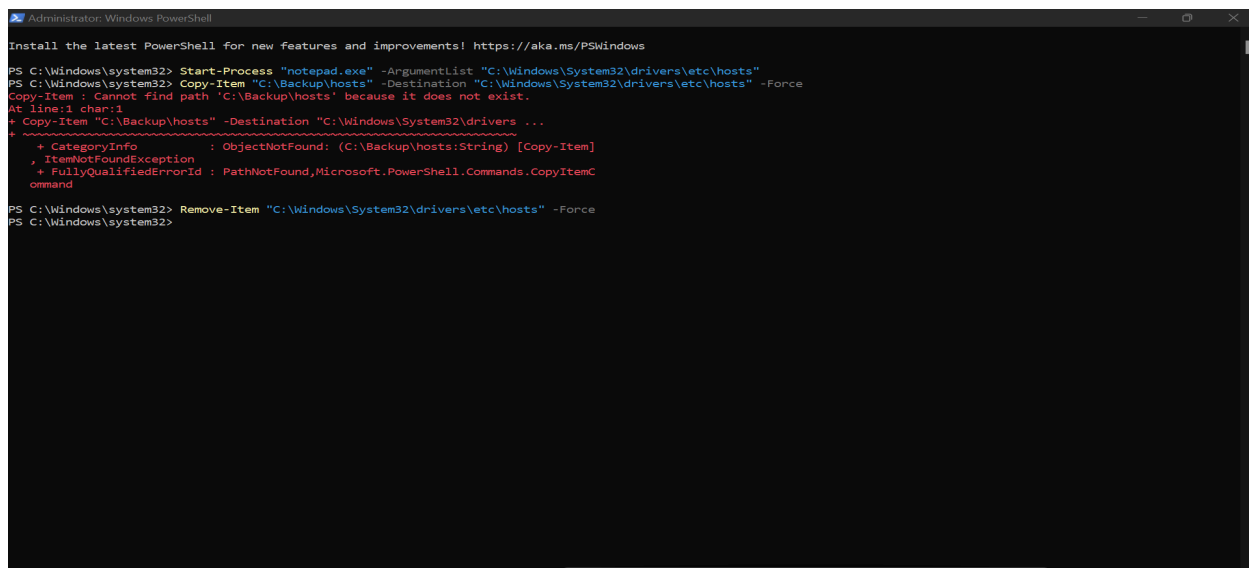
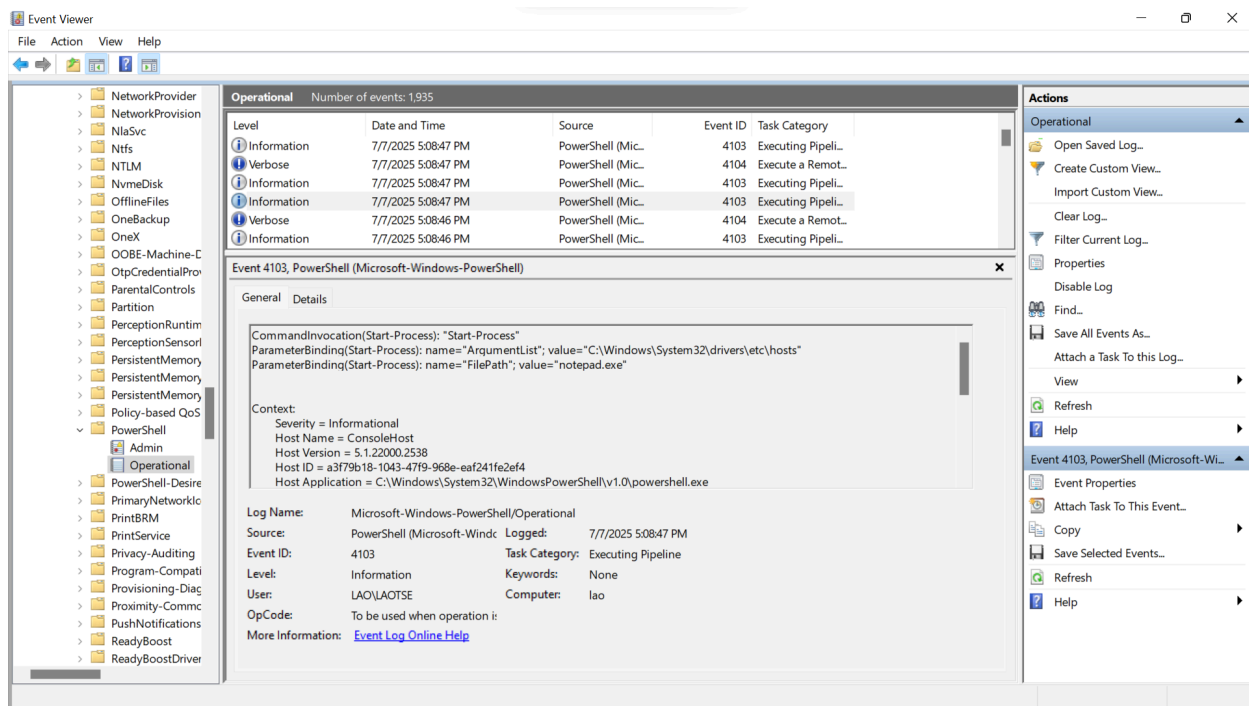


The screenshot shows two windows. On the left is an 'Administrator: Windows PowerShell' terminal window. It displays a message about installing the latest PowerShell and then the command `Start-Process "notepad.exe" -ArgumentList "C:\Windows\System32\drivers\etc\hosts"` being executed. The prompt is `PS C:\Windows\system32>`. On the right is a text editor window titled 'R@In.txt' showing the contents of the hosts file. The file contains a copyright notice for 1993-2009 Microsoft Corp., a description of the file's purpose, and a list of IP addresses mapped to host names, including `102.54.94.97 rhino.acme.com`, `38.25.63.10 x.acme.com`, `127.0.0.1 localhost`, and `::1 localhost`.

```
Administrator: Windows PowerShell
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWin
dows
PS C:\Windows\system32> Start-Process "notepad.exe" -ArgumentList "C:\Windows\System
32\drivers\etc\hosts"
PS C:\Windows\system32>
```

```
File Edit View
# Copyright (c) 1993-2009 Microsoft Corp.
#
# This is a sample HOSTS file used by Microsoft TCP/IP for Windows.
#
# This file contains the mappings of IP addresses to host names. Each
# entry should be kept on an individual line. The IP address should
# be placed in the first column followed by the corresponding host name.
# The IP address and the host name should be separated by at least one
# space.
#
# Additionally, comments (such as these) may be inserted on individual
# lines or following the machine name denoted by a '#' symbol.
#
# For example:
#
#      102.54.94.97      rhino.acme.com      # source server
#      38.25.63.10      x.acme.com         # x client host
#
# localhost name resolution is handled within DNS itself.
#      127.0.0.1        localhost
#      ::1              localhost
```

Ln 13, Col 2 803 characters 100% Windows (CRLF) UTF-8



Key Takeaways:

PowerShell logs (especially Event ID 4103) are critical for detection

PowerShell is both a tool and a threat — logging is essential

Native defenses are effective if properly configured

Day 15: Detecting Outbound Beaconsing (Linux - Kali)

Objective:

Detect and respond to a simulated beaconing connection from a Linux machine to an external IP.

Tasks Performed:

Launched a background curl loop to:

45.13.220.98:443

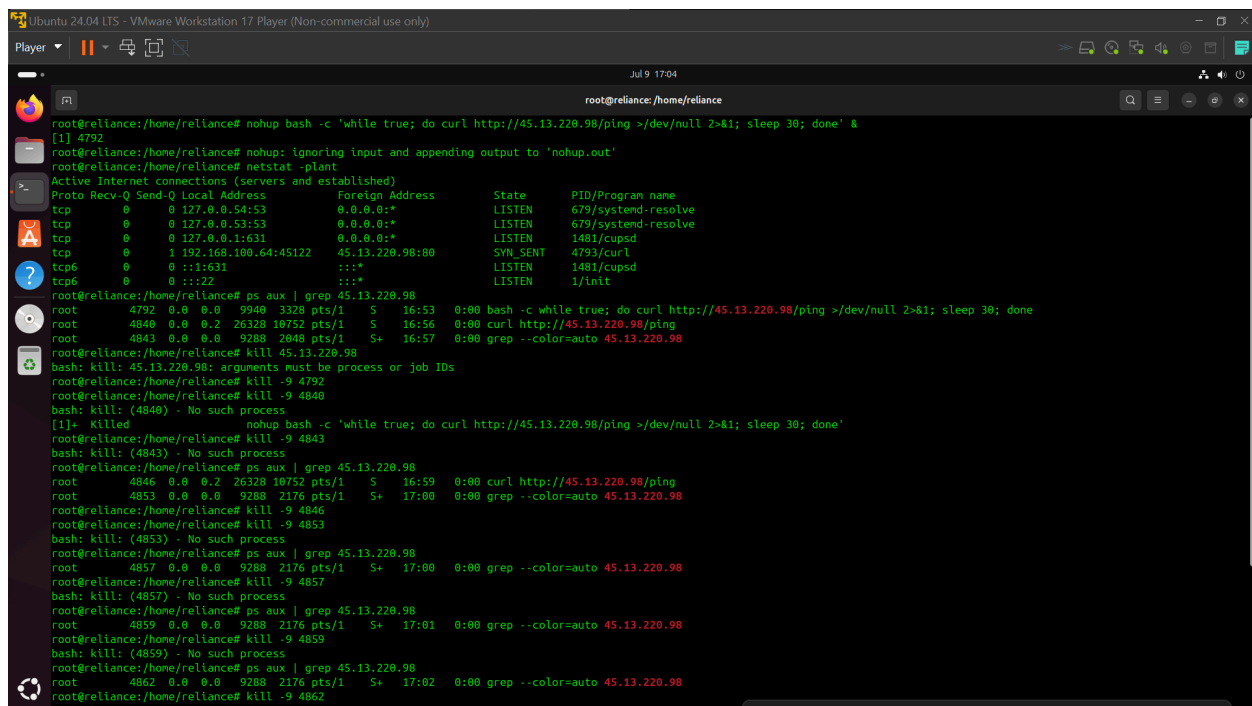
Detected the active connection using netstat -plant and ss -plant

Traced the source using ps aux and lsof

Killed the process using pkill curl

Blocked the IP with:

ufw deny out to 45.13.220.98



```
root@reliance:/home/reliance# nohup bash -c 'while true; do curl http://45.13.220.98/ping >/dev/null 2>&1; sleep 30; done' &
[1] 4792
root@reliance:/home/reliance# nohup: ignoring input and appending output to 'nohup.out'
root@reliance:/home/reliance# netstat -plant
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 127.0.0.1:54:53         0.0.0.0:*               LISTEN      679/systemd-resolve
tcp        0      0 127.0.0.1:631          0.0.0.0:*               LISTEN      1481/cupsd
tcp        0      0 192.168.100.64:45122    45.13.220.98:80         SYN_SENT    4793/curl
tcp6       0      0 :::631                 :::*                   LISTEN      1481/cupsd
tcp6       0      0 :::22                  :::*                   LISTEN      1/init
root@reliance:/home/reliance# ps aux | grep 45.13.220.98
root    4792  0.0  0.0  9940  3328 pts/1    S   16:53    0:00 bash -c while true; do curl http://45.13.220.98/ping >/dev/null 2>&1; sleep 30; done
root    4840  0.0  0.2 26328 10752 pts/1    S   16:56    0:00 curl http://45.13.220.98/ping
root    4843  0.0  0.0  9288  2048 pts/1    S+  16:57    0:00 grep --color=auto 45.13.220.98
root@reliance:/home/reliance# kill 45.13.220.98
bash: kill: 45.13.220.98: arguments must be process or job IDs
root@reliance:/home/reliance# kill -9 4792
root@reliance:/home/reliance# kill -9 4840
bash: kill: (4840) - No such process
[1]+  Killed                  nohup bash -c 'while true; do curl http://45.13.220.98/ping >/dev/null 2>&1; sleep 30; done'
root@reliance:/home/reliance# kill -9 4843
bash: kill: (4843) - No such process
root@reliance:/home/reliance# ps aux | grep 45.13.220.98
root    4846  0.0  0.2 26328 10752 pts/1    S   16:59    0:00 curl http://45.13.220.98/ping
root    4853  0.0  0.0  9288  2176 pts/1    S+  17:00    0:00 grep --color=auto 45.13.220.98
root@reliance:/home/reliance# kill -9 4846
root@reliance:/home/reliance# kill -9 4853
bash: kill: (4853) - No such process
root@reliance:/home/reliance# ps aux | grep 45.13.220.98
root    4857  0.0  0.0  9288  2176 pts/1    S+  17:00    0:00 grep --color=auto 45.13.220.98
root@reliance:/home/reliance# kill -9 4857
bash: kill: (4857) - No such process
root@reliance:/home/reliance# ps aux | grep 45.13.220.98
root    4859  0.0  0.0  9288  2176 pts/1    S+  17:01    0:00 grep --color=auto 45.13.220.98
root@reliance:/home/reliance# kill -9 4859
bash: kill: (4859) - No such process
root@reliance:/home/reliance# ps aux | grep 45.13.220.98
root    4862  0.0  0.0  9288  2176 pts/1    S+  17:02    0:00 grep --color=auto 45.13.220.98
root@reliance:/home/reliance# kill -9 4862
bash: kill: (4862) - No such process
```

```
root@reliance:/home/reliance# ps aux | grep 45.13.220.98
root 4792 0.0 0.0 9940 3328 pts/1 S 16:53 0:00 bash -c while true; do curl http://45.13.220.98/ping >/dev/null 2>&1; sleep 30; done
root 4840 0.0 0.2 26328 10752 pts/1 S 16:56 0:00 curl http://45.13.220.98/ping
root 4843 0.0 0.0 9288 2048 pts/1 S+ 16:57 0:00 grep --color=auto 45.13.220.98
root@reliance:/home/reliance# kill 45.13.220.98
bash: kill: 45.13.220.98: arguments must be process or job IDs
root@reliance:/home/reliance# kill -9 4792
root@reliance:/home/reliance# kill -9 4840
bash: kill: (4840) - No such process
root@reliance:/home/reliance# nohup bash -c 'while true; do curl http://45.13.220.98/ping >/dev/null 2>&1; sleep 30; done'
root@reliance:/home/reliance# kill -9 4843
bash: kill: (4843) - No such process
root@reliance:/home/reliance# ps aux | grep 45.13.220.98
root 4846 0.0 0.2 26328 10752 pts/1 S 16:59 0:00 curl http://45.13.220.98/ping
root 4853 0.0 0.0 9288 2176 pts/1 S+ 17:00 0:00 grep --color=auto 45.13.220.98
root@reliance:/home/reliance# kill -9 4846
root@reliance:/home/reliance# kill -9 4853
bash: kill: (4853) - No such process
root@reliance:/home/reliance# ps aux | grep 45.13.220.98
root 4857 0.0 0.0 9288 2176 pts/1 S+ 17:00 0:00 grep --color=auto 45.13.220.98
root@reliance:/home/reliance# kill -9 4857
bash: kill: (4857) - No such process
root@reliance:/home/reliance# ps aux | grep 45.13.220.98
root 4859 0.0 0.0 9288 2176 pts/1 S+ 17:01 0:00 grep --color=auto 45.13.220.98
root@reliance:/home/reliance# kill -9 4859
bash: kill: (4859) - No such process
root@reliance:/home/reliance# ps aux | grep 45.13.220.98
root 4862 0.0 0.0 9288 2176 pts/1 S+ 17:02 0:00 grep --color=auto 45.13.220.98
root@reliance:/home/reliance# kill -9 4862
bash: kill: (4862) - No such process
root@reliance:/home/reliance# ufw deny out to 45.13.220.98
Rule added
root@reliance:/home/reliance# ufw status
Status: active

To Action From
--
Anywhere DENY 192.168.191.18
45.13.220.98 DENY OUT Anywhere
root@reliance:/home/reliance#
```

Key Takeaway

Outbound connections should be monitored as closely as inbound

Tools like netstat, ss, and ufw provide fast visibility and control

Egress filtering helps prevent data leaks and C2 activity

SUMMARY

These five labs reinforced the core principles of incident response using native tools across both Linux and Windows environments. From brute-force detection to process tracing and cron job cleanup, I was able to simulate attacker behavior and apply the full incident response lifecycle without relying on SIEM or EDR.

I deepened my skills in:

Log analysis (Event Viewer, PowerShell, cron logs)

Threat detection (RDP brute force, beaconing, persistence)

Command-line containment and firewall control

Documentation and response in line with NIST SP 800-61